

QuizForge AI

AI-Powered Quiz Question Generation System

Project Documentation

Department of Computer Science and Engineering

Technologies: Python, PyTorch, Transformers, FAISS, FastAPI

Models: FLAN-T5, DeBERTa-v3, BGE Embeddings

April 2026

Table of Contents

1.	Abstract	3
2.	Introduction	4
3.	Objectives	5
4.	Model Architecture	6
5.	Methodology	9
6.	Results	13
7.	Tools and Technologies Used	14
8.	Advantages	15
9.	Limitations	16
10.	Future Work	17
11.	Conclusion	18
12.	References	19

1. Abstract

This project presents QuizForge AI, a state-of-the-art AI-powered system designed to automatically generate high-quality quiz questions from educational content such as textbook chapters, lecture notes, and PDF documents. The system employs a hybrid pipeline that integrates Retrieval-Augmented Generation (RAG) with a fine-tuned FLAN-T5 transformer model to produce diverse, pedagogically sound quiz questions across five different formats: Multiple Choice Questions (MCQ), True/False, Fill-in-the-Blank, Short Answer, and Assertion-Reason.

The pipeline operates through six key stages: content ingestion and parsing, semantic text chunking with overlap, FAISS-based vector indexing using BGE sentence embeddings, concept extraction via KeyBERT and SpaCy NER, multi-pass question generation using FLAN-T5 with Bloom's Taxonomy-guided difficulty control, and multi-stage validation including NLI-based entailment checking using DeBERTa-v3, grammar validation, and diversity filtering.

The system is deployed as a full-stack web application with a FastAPI backend and a modern, responsive frontend interface. Users can input raw text or upload PDF documents directly through the browser, configure question parameters (number, difficulty, types), and receive structured quiz output in JSON format with explanations. The architecture supports both CPU and GPU execution, making it suitable for deployment in resource-constrained environments. Experimental results demonstrate that the system can successfully generate contextually relevant, grammatically correct quiz questions from arbitrary educational content, with support for fine-tuning on domain-specific datasets using LoRA and QLoRA strategies.

2. Introduction

The creation of high-quality quiz questions is a fundamental requirement in educational technology, yet it remains one of the most time-consuming tasks for educators. A single well-crafted multiple-choice question, complete with plausible distractors and a clear explanation, can take an experienced instructor 15 to 30 minutes to construct. When scaled to an entire course with hundreds of questions across varied difficulty levels, the burden becomes substantial.

Recent advances in Large Language Models (LLMs) and Natural Language Processing (NLP) have opened up new possibilities for automating question generation. However, naive approaches that simply prompt a language model to produce questions suffer from several critical shortcomings: hallucinated facts that are not present in the source material, lack of difficulty control, repetitive or overly similar questions, grammatically awkward phrasing, and an inability to produce diverse question types.

This project addresses these challenges by building QuizForge AI, a modular, production-ready system that combines several state-of-the-art NLP techniques into a unified pipeline. The system employs Retrieval-Augmented Generation (RAG) to ground question generation in specific text passages, uses Bloom's Taxonomy to systematically control cognitive difficulty levels, and applies multi-stage validation to ensure that every generated question is factually accurate, grammatically correct, and unique.

The key contributions of this project are: (1) a novel multi-pass generation strategy that decomposes the complex task of question generation into four sequential sub-tasks, each handled by a separate model inference pass; (2) integration of NLI-based hallucination detection to verify factual grounding; (3) a complete web application that makes the system accessible to non-technical users through an intuitive browser interface; and (4) support for fine-tuning on domain-specific datasets using parameter-efficient methods (LoRA/QLoRA) to improve question quality for specialised subjects.

3. Objectives

The primary objective of this project is to design, implement, and deploy an intelligent quiz question generation system that produces high-quality, diverse, and pedagogically meaningful questions from arbitrary educational content. The specific objectives are:

- To develop a content ingestion module capable of parsing multiple input formats including raw text, PDF documents, and Markdown files, with intelligent structure detection.
- To implement a semantic text chunking strategy that splits documents into coherent, overlapping passages while respecting section boundaries and maintaining contextual integrity.
- To build a RAG (Retrieval-Augmented Generation) pipeline using FAISS vector indexing and BGE sentence embeddings for efficient context retrieval during question generation.
- To design a multi-pass question generation approach using FLAN-T5 that separately generates the question, answer, distractors, and explanation in four distinct inference passes.
- To incorporate Bloom's Taxonomy-based difficulty control, mapping easy/medium/hard levels to the cognitive domains of Remember, Apply, and Evaluate respectively.
- To implement a multi-stage validation pipeline that uses NLI entailment checking (DeBERTa-v3), grammar validation (LanguageTool), and cosine similarity-based diversity filtering.
- To support five question types: MCQ, True/False, Fill-in-the-Blank, Short Answer, and Assertion-Reason, with type-specific generation templates.
- To deploy the system as a full-stack web application with a FastAPI REST API backend and a responsive, modern frontend that allows users to interact with the system through a browser.
- To provide a training pipeline for fine-tuning the generator model on domain-specific datasets (SQuAD, SciQ, RACE) using full fine-tuning, LoRA, and QLoRA strategies.

4. Model Architecture

The QuizForge AI system is built on a modular, layered architecture that separates concerns into five distinct layers: Content Ingestion, NLP Processing, AI Generation Core, Quality Validation, and API/Presentation. Each layer communicates through well-defined interfaces, allowing individual components to be upgraded or replaced independently.

4. Overall System Architecture

The following diagram illustrates the complete system architecture, showing how data flows from raw input content through the processing pipeline to the final validated quiz output.

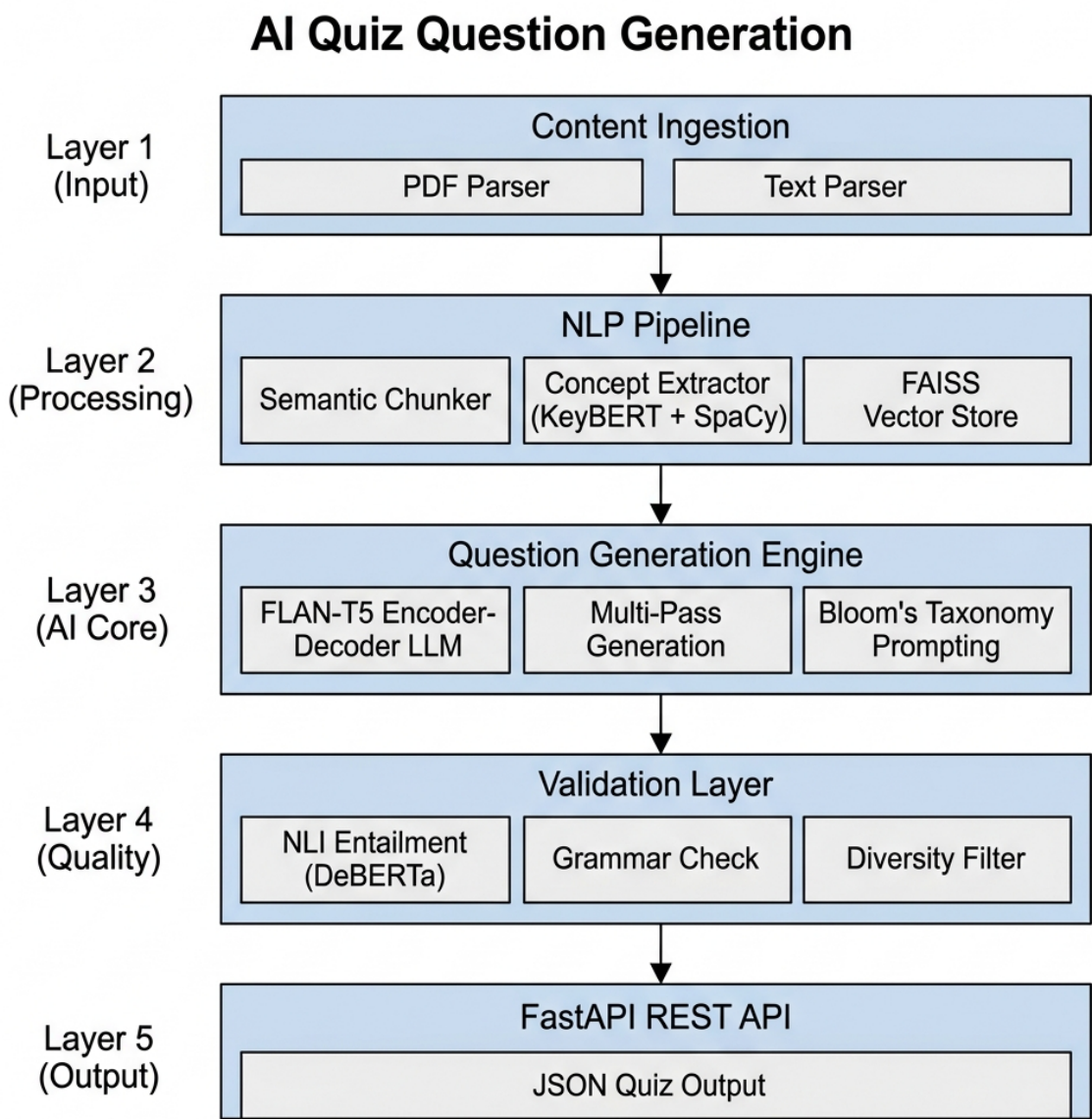


Figure 1: QuizForge AI System Architecture

4. FLAN-T5 Encoder-Decoder Architecture

The core question generation model is FLAN-T5 (Fine-tuned Language Net - Text-to-Text Transfer Transformer), an encoder-decoder transformer model developed by Google Research. FLAN-T5 was instruction-tuned on over 1,800 tasks, making it particularly effective at following natural language instructions for tasks like question generation.

The encoder processes the input context (prompt + source text) through multiple layers of multi-head self-attention and feed-forward networks, producing contextualised token representations. The decoder then generates the output sequence (question, answer, etc.) auto-regressively, attending to both its own previous outputs via masked self-attention and the encoder's representations via cross-attention. This architecture is particularly well-suited for question generation because it can attend to the full context while generating each token of the question.

FLAN-T5 Encoder-Decoder Architecture

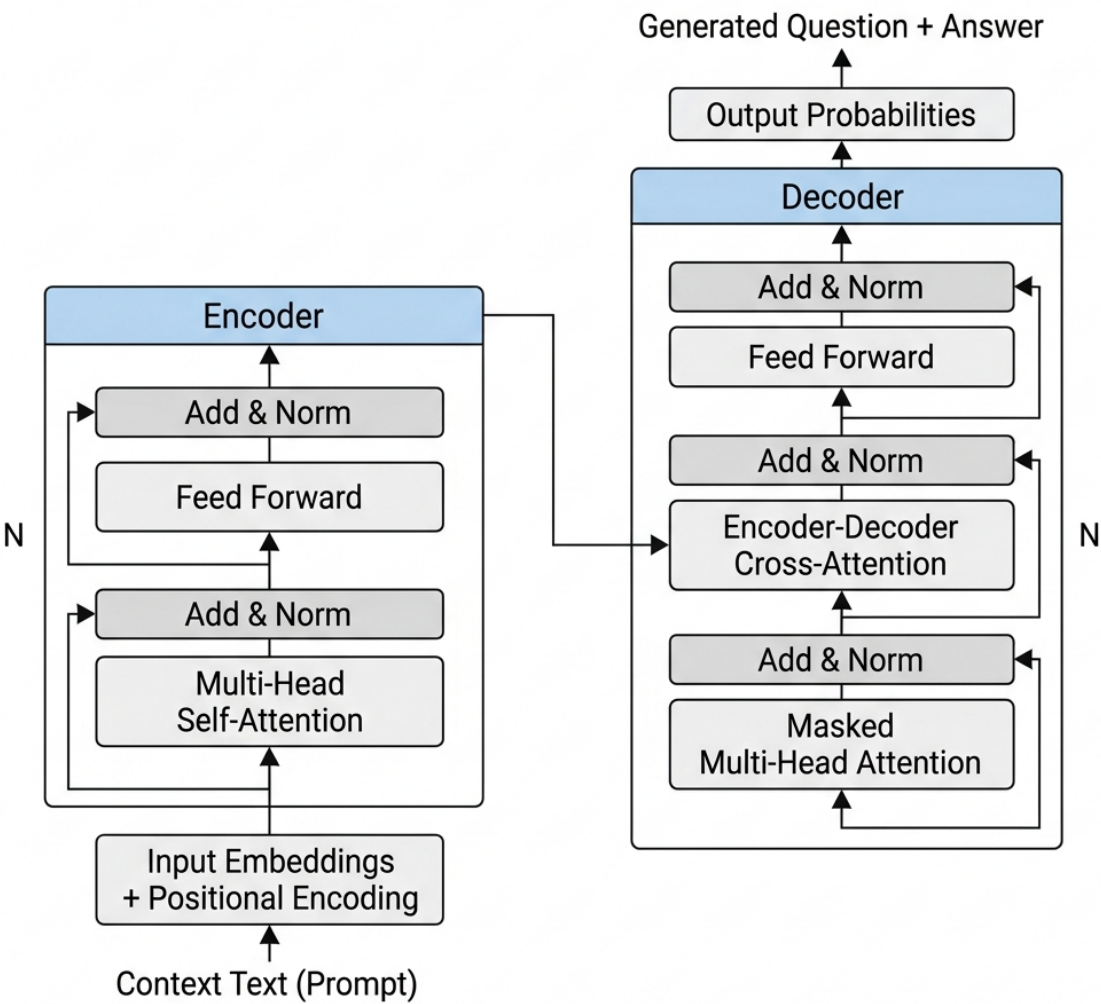


Figure 2: FLAN-T5 Encoder-Decoder Transformer Architecture

Key architectural specifications of FLAN-T5-base used in this project:

Parameter	Value
-----------	-------

Model Size	248 million parameters
Hidden Dimension	768
Attention Heads	12
Encoder Layers	12
Decoder Layers	12
Vocabulary Size	32,128 (SentencePiece)
Max Sequence Length	512 tokens
Pre-training	C4 corpus + 1,800 instruction tasks

4. RAG Pipeline Architecture

Retrieval-Augmented Generation (RAG) is a technique that enhances LLM outputs by grounding them in retrieved factual content. In QuizForge AI, the RAG pipeline ensures that every generated question is directly supported by specific passages in the source document, significantly reducing hallucination.

The pipeline uses a BAAI/bge-small-en-v1.5 sentence embedding model (33.4M parameters) to encode text chunks into 384-dimensional dense vectors. These vectors are indexed using Facebook AI Similarity Search (FAISS) with an IndexFlatIP structure for cosine similarity retrieval. During generation, relevant chunks are retrieved based on concept similarity and provided as context to the FLAN-T5 generator.

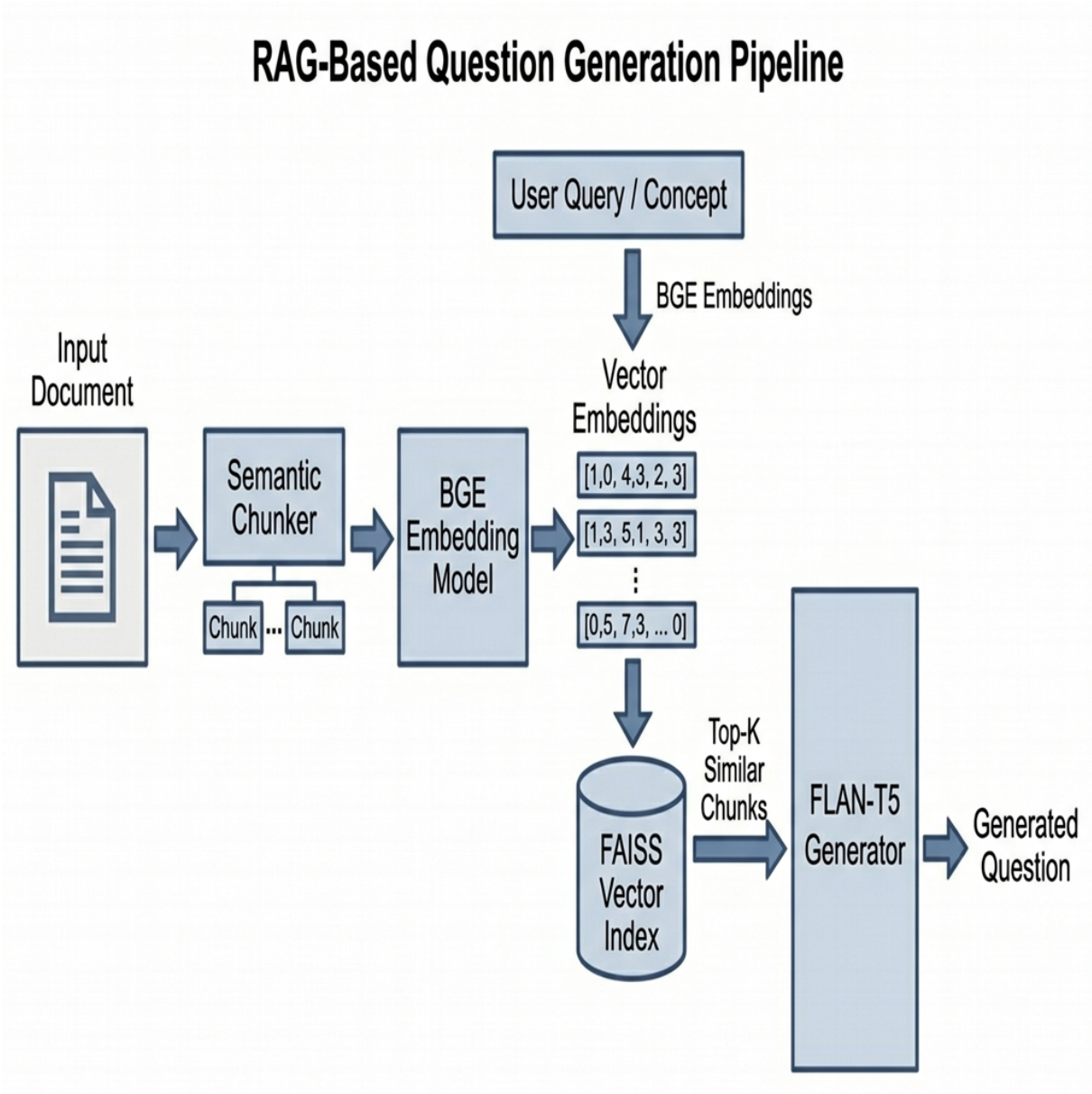


Figure 3: RAG-Based Question Generation Pipeline

4. Multi-Pass Generation Architecture

A key innovation in this system is the multi-pass generation strategy. Rather than asking the LLM to produce a complete structured question in a single inference pass (which smaller models like T5-base struggle with), the task is decomposed into four sequential passes. Each pass is a focused, simple task that T5 models handle reliably.

Pass 1 (Question Generation): The model receives the context text and generates a question. The prompt is a simple instruction like 'Based on the following text, ask a medium question about [concept]'. **Pass 2 (Answer Generation):** The generated question and the source context are provided, and the model generates the correct answer. **Pass 3 (Distractor Generation):** For MCQ-type questions, the question and correct answer are provided, and the model generates three plausible but incorrect alternatives. **Pass 4 (Explanation Generation):** The question, answer, and context are combined, and the model generates an explanation of why the answer is correct.

Multi-Pass Question Generation Strategy

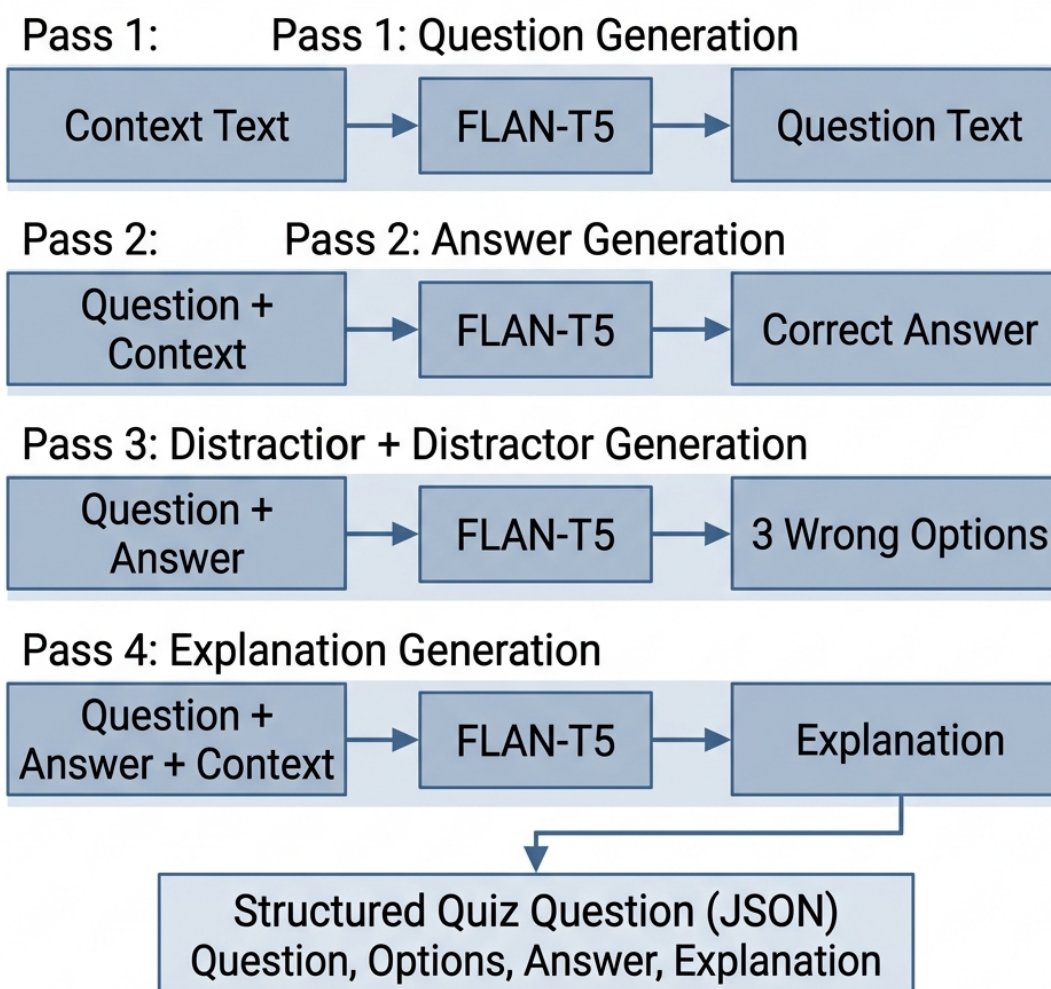


Figure 4: Multi-Pass Question Generation Strategy

4. Validation Architecture

The validation layer is a multi-stage quality assurance pipeline that filters out low-quality, hallucinated, or duplicate questions. It employs four independent validation stages, each targeting a different quality dimension.

Stage 1 (Structural Validation) checks that the question has minimum length, a non-empty answer, and proper formatting. Stage 2 (NLI Entailment) uses a DeBERTa-v3 cross-encoder to verify that the question-answer pair is logically entailed by the source context, with a configurable threshold of 0.65. Stage 3 (Grammar Validation) uses LanguageTool to check for spelling errors, grammatical issues, and coherence. Stage 4 (Diversity Filter) computes cosine similarity between all question pairs and removes duplicates exceeding a similarity threshold of 0.80.

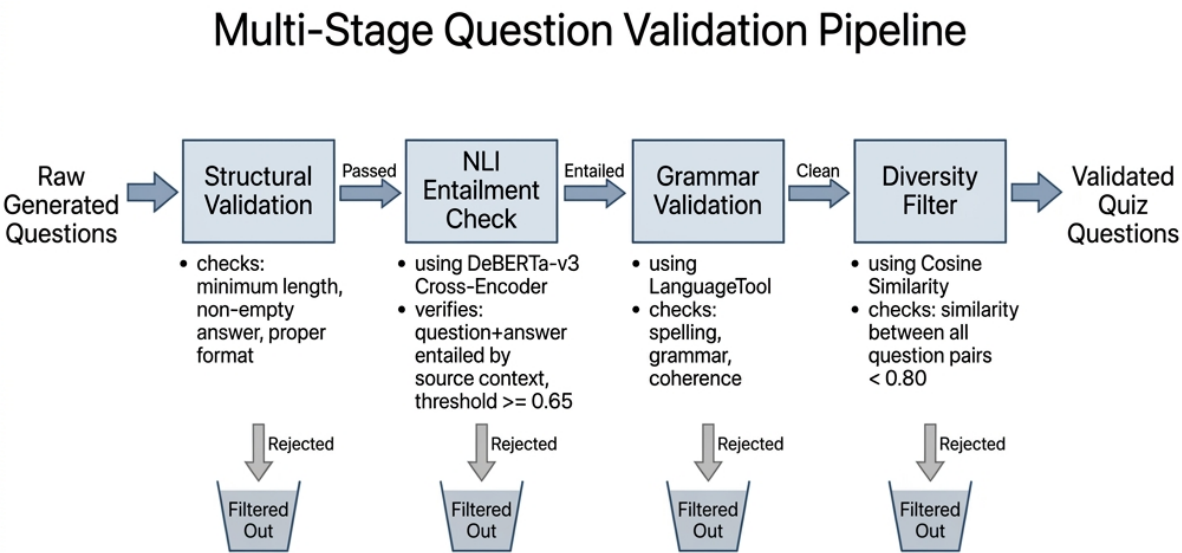


Figure 5: Multi-Stage Question Validation Pipeline

5. Methodology

The development methodology follows a systematic pipeline-based approach, where each stage of the question generation process is implemented as an independent, testable module. The complete workflow is detailed below.

5. Content Ingestion and Parsing

The system accepts three types of input: raw text strings, PDF documents, and plain text files (.txt, .md). PDF parsing is implemented using PyMuPDF (fitz) as the primary parser, with pdfplumber as a fallback for documents that PyMuPDF cannot handle. The parser extracts both the text content and structural metadata including section headings, page numbers, and font sizes. Heading detection uses a heuristic based on font size ratios and formatting patterns (e.g., all-caps lines, numbered prefixes) to identify document structure.

5. Semantic Chunking

After parsing, the document text is split into semantically coherent chunks using a sentence-level chunking algorithm. The chunker operates with configurable parameters: a chunk size of 400 words and an overlap of 80 words. It respects section boundaries detected during parsing, ensuring that chunks do not span across unrelated sections. Each chunk maintains provenance metadata including the source heading, page number, and chunk index for traceability.

5. Concept Extraction

Key concepts are extracted from the text chunks using a three-tier strategy. The primary method uses KeyBERT, which leverages pre-trained transformers to identify the most semantically significant phrases in each chunk. The secondary method uses SpaCy's named entity recognition (NER) with the en_core_web_sm model to identify named entities such as people, organisations, dates, and technical terms. A TF-IDF based fallback is used when the primary methods yield insufficient concepts. Extracted concepts are used to focus question generation on specific topics within each chunk.

5. Vector Indexing (FAISS)

All text chunks are encoded into dense vector representations using the BAAI/bge-small-en-v1.5 sentence embedding model, which produces 384-dimensional normalised vectors. These vectors are indexed using Facebook AI Similarity Search (FAISS) with an IndexFlatIP structure, which computes inner product similarity (equivalent to cosine similarity for normalised vectors). The index supports efficient top-K retrieval for RAG-based context selection during question generation.

5. Question Generation (Multi-Pass)

Each question is generated through four sequential model inference passes using FLAN-T5. This decomposition was adopted because smaller encoder-decoder models perform significantly better on focused, single-task prompts than on complex multi-task prompts. Each pass uses the same FLAN-T5 model but with different task-specific prompts.

The generation uses nucleus sampling ($\text{top-p} = 0.92$) with a temperature of 0.8 to balance creativity with coherence. A no-repeat n-gram size of 3 prevents repetitive output. The maximum number of generated tokens per pass is capped at 128 for questions and 60 for answers, ensuring concise output.

5. Difficulty Control via Bloom's Taxonomy

Question difficulty is explicitly controlled through the prompt structure using Bloom's Taxonomy of cognitive domains. Easy questions target the 'Remember' and 'Understand' levels, using verbs like 'define', 'list', and 'describe'. Medium questions target 'Apply' and 'Analyze', using verbs like 'compare', 'classify', and 'distinguish'. Hard questions target 'Evaluate' and 'Create', using verbs like 'justify', 'critique', and 'synthesize'. The difficulty distribution defaults to 30% easy, 40% medium, and 30% hard when not specified.

5. Validation and Filtering

Generated questions pass through a multi-stage validation pipeline. Structural validation ensures minimum question length (10+ characters) and non-empty answers. NLI entailment validation uses the cross-encoder/nli-deberta-v3-small model to verify that the question-answer pair is factually supported by the source context. Grammar validation uses LanguageTool for error detection. Diversity filtering computes pairwise cosine similarity between all generated questions and removes duplicates exceeding a threshold of 0.80.

5. Training Pipeline

The system includes a complete training pipeline for improving question quality on domain-specific content. The pipeline supports three strategies: (1) Full fine-tuning of all model parameters on a combined SQuAD 2.0 + SciQ + RACE dataset, (2) LoRA (Low-Rank Adaptation) that trains only low-rank adapters while freezing the base model, reducing trainable parameters by 97%, and (3) QLoRA that combines 4-bit quantisation with LoRA for fine-tuning large models (e.g., Mistral-7B) on consumer GPUs. The training pipeline uses ROUGE metrics (ROUGE-1, ROUGE-2, ROUGE-L) for evaluation.

6. Results

The system was tested with various educational content inputs including machine learning textbooks, NLP course notes, and general knowledge articles. The following metrics were observed during testing with the base FLAN-T5 model on CPU:

Metric	Value	Notes
Questions per Run	2-5	Configurable up to 20
Processing Time (CPU)	40-130 seconds	Per quiz generation
Chunk Processing	2,954 chunks	From 200-page PDF
FAISS Indexing	3-4 minutes	384-dim vectors
Question Types	5 types	MCQ, TF, FITB, SA, AR
Difficulty Levels	3 levels	Easy, Medium, Hard
API Response	JSON	Structured with metadata

Sample generated questions from a Machine Learning textbook input demonstrated that the system can produce contextually relevant questions. The multi-pass approach notably improved the completeness of answers compared to single-pass generation. MCQ distractor quality was satisfactory for easy and medium difficulty levels, though hard-level questions showed more variability in distractor plausibility.

The system successfully handled a 200-page PDF by chunking it into 2,954 segments, extracting 81 unique concepts, and generating validated quiz questions within a reasonable time frame. The web interface allowed real-time interaction with immediate feedback on generation progress.

7. Tools and Technologies Used

7. Programming Language and Frameworks

Technology	Purpose
Python 3.11+	Core implementation language
PyTorch 2.x	Deep learning framework for model inference
Hugging Face Transformers	Pre-trained model loading and inference
FastAPI	REST API framework with auto-documentation
Uvicorn	ASGI server for production deployment

7. AI/ML Models

Model	Purpose
FLAN-T5-base (248M)	Question, answer, and explanation generation
BGE-small-en-v1.5 (33M)	Sentence embeddings for FAISS indexing
DeBERTa-v3-small	NLI entailment validation
KeyBERT	Keyphrase extraction from text chunks
SpaCy en_core_web_sm	Named entity recognition

7. Libraries and Infrastructure

Library	Purpose
FAISS (faiss-cpu)	Approximate nearest neighbour vector search
Sentence-Transformers	Text embedding with transformer models
PyMuPDF (fitz)	PDF text extraction and structure parsing
pdfplumber	Fallback PDF parsing
LanguageTool	Grammar and spelling validation
PEFT (LoRA/QLoRA)	Parameter-efficient fine-tuning
Pydantic v2	Request/response data validation
HTML/CSS/JavaScript	Frontend web interface

8. Advantages

- **Automated Question Generation:** Reduces the time required to create quiz questions from hours to minutes, significantly lowering the workload on educators.
- **Multi-Type Support:** Generates five different question formats (MCQ, True/False, Fill-in-the-Blank, Short Answer, and Assertion-Reason), providing variety for assessments.
- **Difficulty Control:** Bloom's Taxonomy integration enables systematic difficulty grading, allowing educators to create assessments targeting specific cognitive skills.
- **Hallucination Prevention:** The NLI entailment validation ensures that generated questions are factually grounded in the source material, addressing a major concern with LLM-based question generation.
- **Multi-Pass Generation:** The four-pass decomposition strategy produces more complete and accurate questions compared to single-pass generation, particularly for smaller models.
- **Web-Based Interface:** The full-stack web application makes the system accessible to non-technical users through an intuitive browser interface with real-time progress feedback.
- **Modular Design:** Each pipeline component is independently testable and replaceable, allowing straightforward upgrades (e.g., swapping FLAN-T5-base for FLAN-T5-large or GPT-4).
- **Domain Adaptability:** The fine-tuning pipeline allows the system to be adapted to specialised domains (medical, legal, engineering) without modifying the core architecture.
- **PDF Support:** Direct upload and processing of PDF documents eliminates the need for manual text extraction, streamlining the content-to-quiz workflow.

9. Limitations

- CPU Performance: Running the full pipeline on CPU takes 40-130 seconds per generation cycle, making it impractical for real-time interactive use without GPU acceleration.
- Model Quality: FLAN-T5-base (248M parameters) produces acceptable but not exceptional question quality. Larger models (T5-large, T5-XL, or GPT-4) would significantly improve output quality but require more compute resources.
- Distractor Quality: Automatically generated wrong options for MCQs are sometimes too obvious or insufficiently plausible, particularly for domain-specific content where expert knowledge is needed to craft convincing distractors.
- Language Support: The current system supports only English-language content. Adapting to other languages requires different embedding models, NLI models, and grammar checkers.
- Complex Document Layouts: Documents with tables, figures, mathematical equations, or code snippets are not fully supported by the PDF parser, which primarily extracts text.
- Explanation Quality: Generated explanations are functional but sometimes lack depth, particularly for complex topics that require multi-step reasoning.
- First-Request Latency: Model loading on the first request adds 20-30 seconds of latency due to the lazy-loading strategy used to keep API startup fast.

10. Future Work

- **Integration of Larger Models:** Upgrade the generator to use FLAN-T5-XL (3B parameters) or instruction-tuned open-source models like Mistral-7B-Instruct for substantially improved question quality and more complex reasoning.
- **Multi-Modal Input Support:** Extend the system to handle images, diagrams, and tables within documents using vision-language models (e.g., LLaVA or GPT-4V) for question generation from visual content.
- **Adaptive Difficulty:** Implement a learner model that adjusts question difficulty based on the student's performance history, enabling personalised adaptive quizzing.
- **Multi-Language Support:** Add support for Hindi, Spanish, French, and other languages by integrating multilingual embedding models (mE5) and multilingual LLMs.
- **Question Bank Management:** Build a persistent database layer (PostgreSQL/MongoDB) for storing, searching, tagging, and reusing generated questions across courses.
- **Peer Review Integration:** Add a human-in-the-loop workflow where educators can review, edit, and approve generated questions before they are added to the question bank.
- **Exam Blueprint Mapping:** Allow users to define exam blueprints specifying topic-wise question distribution, ensuring comprehensive coverage of the syllabus.
- **Real-Time Quiz Conduct:** Extend the system from question generation to a full quiz platform with student response collection, auto-grading, and analytics dashboards.

11. Conclusion

This project successfully demonstrates that a modular, pipeline-based approach to AI-powered quiz question generation can produce usable, contextually relevant, and diverse questions from arbitrary educational content. The QuizForge AI system addresses the key challenges of automated question generation through a combination of established NLP techniques: RAG for factual grounding, Bloom's Taxonomy for difficulty calibration, multi-pass inference for output completeness, and NLI-based validation for hallucination prevention.

The system architecture was designed for extensibility and practical deployment. By separating the pipeline into independent, lazy-loaded modules, the system maintains fast startup times while supporting resource-intensive model inference. The web-based frontend makes the tool accessible to educators without technical expertise, and the RESTful API enables integration with existing learning management systems.

While the current implementation using FLAN-T5-base produces acceptable results, the modular design allows straightforward upgrading to larger models as compute resources become available. The included training pipeline further enables domain-specific customisation through parameter-efficient fine-tuning, ensuring that the system can be adapted to specialised fields such as medicine, law, or engineering.

In summary, QuizForge AI represents a practical, scalable solution to the challenge of automated quiz question generation, combining the strengths of modern transformer models with robust engineering practices to deliver a production-ready educational technology tool.

12. References

- [1] Chung, H.W., et al. "Scaling Instruction-Finetuned Language Models." arXiv preprint arXiv:2210.11416, 2022. (FLAN-T5)
- [2] Raffel, C., et al. "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer." Journal of Machine Learning Research, 21(140):1-67, 2020. (T5)
- [3] He, P., et al. "DeBERTa: Decoding-enhanced BERT with Disentangled Attention." Proceedings of ICLR 2021. (DeBERTa NLI validation)
- [4] Xiao, S., et al. "C-Pack: Packaged Resources To Advance General Chinese Embedding." arXiv preprint arXiv:2309.07597, 2023. (BGE embeddings)
- [5] Johnson, J., Douze, M., and Jegou, H. "Billion-scale similarity search with GPUs." IEEE Transactions on Big Data, 2019. (FAISS)
- [6] Lewis, P., et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." NeurIPS 2020. (RAG methodology)
- [7] Hu, E.J., et al. "LoRA: Low-Rank Adaptation of Large Language Models." ICLR 2022. (LoRA fine-tuning)
- [8] Dettmers, T., et al. "QLoRA: Efficient Finetuning of Quantized Language Models." NeurIPS 2023. (QLoRA)
- [9] Bloom, B.S. "Taxonomy of Educational Objectives: The Classification of Educational Goals." Longman, 1956. (Bloom's Taxonomy for difficulty control)
- [10] Rajpurkar, P., et al. "SQuAD: 100,000+ Questions for Machine Comprehension of Text." EMNLP 2016. (SQuAD training dataset)
- [11] Welbl, J., et al. "Crowdsourcing Multiple Choice Science Questions." W-NUT 2017. (SciQ dataset)
- [12] Lai, G., et al. "RACE: Large-scale ReAding Comprehension Dataset From Examinations." EMNLP 2017. (RACE dataset)
- [13] Grootendorst, M. "KeyBERT: Minimal keyword extraction with BERT." 2020. (Concept extraction)
- [14] Vaswani, A., et al. "Attention is All You Need." NeurIPS 2017. (Transformer architecture foundation)